EX 7: House Price Prediction using Linear Regression

Aim

To develop a predictive model using **Linear Regression** to estimate house prices based on various features such as area, bedrooms, bathrooms, and amenities.

Algorithm

1. Import required libraries (Pandas, NumPy, Matplotlib, Seaborn, Sklearn).
2. Load the housing dataset.
3. Perform Exploratory Data Analysis (EDA):
    - Display dataset structure (head, info, describe)
    - Check missing values
4. Perform outlier analysis using boxplots.
5. Visualize relationships using pairplot.
6. Preprocess the data:
    - Convert categorical variables into numerical using one-hot encoding.
7. Split the dataset into training and testing sets.
8. Train the Linear Regression model using training data.
9. Predict house prices using test data.
10. Evaluate the model using:
    - Mean Squared Error (MSE)
    - R-squared score
11. Visualize:
    - Actual vs Predicted values
    - Residual plot
12. Interpret results and conclude.

Code

```
# 1. Import Libraries

import pandas as pd

import numpy as np
```

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load Dataset
df = pd.read_csv("Housing.csv")

# 3. EDA
print(df.head())
print(df.info())
print(df.describe())

# 4. Missing Values
print(df.isnull().sum())

# 5. Outlier Analysis
fig, axs = plt.subplots(2, 3, figsize=(10,5))
sns.boxplot(df['price'], ax=axs[0,0])
sns.boxplot(df['area'], ax=axs[0,1])
sns.boxplot(df['bedrooms'], ax=axs[0,2])
sns.boxplot(df['bathrooms'], ax=axs[1,0])
sns.boxplot(df['stories'], ax=axs[1,1])
sns.boxplot(df['parking'], ax=axs[1,2])
plt.tight_layout()
```

```
# 6. Pairplot
sns.pairplot(df)
plt.show()

# 7. Data Preprocessing
features = ['mainroad','guestroom','basement','hotwaterheating','airconditioning','prefarea']
df_encoded = pd.get_dummies(df[features], drop_first=True)

df_encoded['price'] = df['price']

# 8. Splitting Data
X = df_encoded.drop('price', axis=1)
y = df_encoded['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 9. Model Building
model = LinearRegression()
model.fit(X_train, y_train)

# 10. Prediction
y_pred = model.predict(X_test)

# 11. Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
```

```
print("R-squared:", r2)


# 12. Visualization
plt.figure(figsize=(8,5))
plt.scatter(y_test, y_pred)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted")
plt.show()


# Residual Plot
plt.figure(figsize=(8,5))
sns.residplot(x=y_pred, y=y_test - y_pred, lowess=True)
plt.axhline(0, linestyle='--')
plt.title("Residual Plot")
plt.show()
```

Result

- **Mean Squared Error (MSE):** 3254189950926.765
- **R-squared Score:** 0.356